React Native

Learning Field Guide

Gautam's Personal Concept Bible

Windows · Expo SDK 54 · Android

This guide covers every concept used while building the BeReal clone app — from absolute zero. Each concept is explained in plain language: what it is, why it exists, what happens in memory, and what breaks if you do it wrong.

#01	**useState**	What memory actually IS. Why variables break.
#02	**useEffect**	Lifecycle, timing, dependency arrays, cleanup.
#03	**Context API**	Sharing data without prop drilling.
#04	**TypeScript**	Interfaces, types, why isAuth crashed you.
#05	**JSX**	What it compiles to. Why no if/else inside it.
#06	**async/await**	Promises without the chaos. try/catch pattern.
#07	**Expo Router**	File-based navigation. push vs replace.

Quick Glossary

Every technical term used in this guide is defined here. Read this first — it makes all the concept explanations significantly clearer.

useState

A React hook that stores a value outside your component function so it survives re-renders. Changing it via the setter function triggers a UI update.

useEffect

A React hook that runs code AFTER a render completes. Used for side effects: data fetching, timers, navigation guards, subscriptions.

Context API

React's built-in system for sharing data across components without passing it as props through every level. createContext + Provider + useContext.

TypeScript

A superset of JavaScript that adds type checking. Catches bugs at compile time (in VS Code) before they crash your app at runtime.

interface

A TypeScript keyword that describes the shape an object must have. Like a contract: if you say something is a User, it must have all the required fields.

JSX

The HTML-looking syntax in your React components. It is NOT HTML — Babel compiles it into React.createElement() function calls before the app runs.

async/await

Syntax for writing Promise-based code that looks synchronous. await pauses a function and waits for a Promise to resolve without freezing the UI.

Promise

An object representing a value that is not available yet. Has 3 states: pending, resolved (success), rejected (failure).

Expo Router

Expo's file-based navigation system. The filename becomes the route. No manual route registration needed.

useRouter

A hook from Expo Router that gives you push(), replace(), back() and other navigation methods.

AuthProvider

A Context Provider component that wraps your app and makes auth data (user, signIn, signUp) available to all screens.

ReactNode

A TypeScript type meaning 'anything React can render' — JSX elements, strings, numbers, arrays, null, or booleans.

Supabase

A Backend-as-a-Service. Provides a PostgreSQL database, authentication, and file storage with a JavaScript SDK.

AsyncStorage

A simple key-value storage system for React Native. Like localStorage but for mobile apps. Used by Supabase to persist login sessions.

#01 useState

What memory actually IS in React

What is this concept?

In normal JavaScript, every variable inside a function is born and dies with that function call. Your React component is just a function that React calls every time the screen needs to update. Every render, your variable resets to its initial value — you lose everything the user typed. useState solves this by storing values OUTSIDE your component in React's internal memory, so they survive across renders.

Why did we need it?

Every form in your app — login, signup, onboarding — captures what the user types. Without useState, every keypress would trigger a re-render and immediately erase what was just typed. We also used it to track loading state (the spinner) and the selected profile image URI.

```
# Why a regular variable completely breaks your UI
// BAD — this resets to "" on EVERY single render
function LoginScreen() {
let email = ""; // created fresh every render
return (
<TextInput
onChangeText={v => { email = v; }}
// Problem 1: updating email does NOT tell React anything changed
// Problem 2: even if re-render happens, email resets to ""
/>
);
}
```

```
# useState — full anatomy with what actually happens
const [email, setEmail] = useState("");
// ^ ^ ^
// current setter function initial value
// value (call this to (used ONCE on first render,
// update + re-render) ignored after that)
// useState returns an array of exactly 2 things.
// The [] is just JavaScript array destructuring:
const pair = useState("");
const email = pair[0]; // same thing, less clean
const setEmail = pair[1]; // same thing, less clean
// What happens when you call setEmail("hello@gmail.com"):
// 1. React saves "hello@gmail.com" in its internal fiber tree
// 2. React marks this component as dirty (needs re-render)
// 3. React calls your component function again
// 4. useState() returns "hello@gmail.com" (not "")
// 5. Your TextInput now shows the saved value
```

From your actual code

```
# signup.tsx — all 3 state variables explained
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [isLoading, setIsLoading] = useState(false);
// isLoading is a boolean — it drives the loading spinner:
{isLoading
? <ActivityIndicator size={24} color="#fff" />
: <Text style={styles.buttonText}>Sign Up</Text>
}
// setIsLoading(true) -> re-render -> spinner appears
// setIsLoading(false) -> re-render -> button text returns
// onboarding.tsx adds image state:
const [profileImage, setProfileImage] = useState<string | null>(null);
// <string | null> = TypeScript: holds a string URI or null
// null -> shows the "+" placeholder circle
// "file:///..." -> shows the actual selected image
```

Critical rule: NEVER mutate state directly

Do NOT write: email = 'something'. That updates the variable but does not tell React anything changed. No re-render. The screen stays stale. ALWAYS use the setter: setEmail('something'). The setter is the trigger.

Data warning: useState updates are asynchronous

After calling setEmail('hello'), you cannot immediately read the new value. console.log(email) right after setEmail() still shows the OLD value. The update happens on the NEXT render. If you need the new value immediately, use the value you passed to the setter directly.

#02 useEffect

Lifecycle, timing, dependency arrays, cleanup

What is this concept?

useEffect lets you run side effects AFTER a render completes. A side effect is anything that interacts with the outside world: fetching data from a server, starting a timer, calling an API, checking auth state. React deliberately keeps these separate from rendering because rendering must be fast and pure.

```
# The 3 forms of useEffect
// FORM 1: runs after EVERY render (almost never what you want)
useEffect(() => {
console.log("runs after every render");
}); // no dependency array
// FORM 2: runs ONCE after first render only
useEffect(() => {
// setup code: check auth, fetch initial data, start listeners
}, []); // empty array = "zero dependencies = run once on mount"
// FORM 3: runs when specific values change
useEffect(() => {
fetchUserProfile(userId);
}, [userId]); // runs on mount AND whenever userId changes
// CLEANUP: returned function runs before next effect or on unmount
useEffect(() => {
const t = setTimeout(() => { doSomething(); }, 1000);
return () => clearTimeout(t); // cancel if component unmounts
}, []);
```

The bug you hit: zombie useEffect in signup.tsx

Zombie useEffect — still in your file right now

You wrote: useEffect(() => { router.push('/(auth)/onboarding'); }, []). The [] means 'run once on mount'. Mount = the instant the signup screen appears. So BEFORE the user types anything, they get teleported to onboarding. This is still in your uploaded signup.tsx. Fix: delete the entire useEffect. Put router.replace() inside handleSignUp after await signUp() succeeds.

```
# Wrong vs correct pattern for post-action navigation
// WRONG: navigation inside useEffect
useEffect(() => {
router.push("/(auth)/onboarding"); // fires on MOUNT not on success!
}, []);
// RIGHT: navigation inside the button handler
const handleSignUp = async () => {
setIsLoading(true);
try {
await signUp(email, password);
router.replace("/(auth)/onboarding"); // fires after success
} catch (error: any) {
Alert.alert("Error", error.message);
} finally {
setIsLoading(false);
}
};
```

The setTimeout(fn, 0) pattern in your _layout.tsx

In your _layout.tsx the navigation call is wrapped in setTimeout with 0 milliseconds. This seems pointless but it is critical. JavaScript uses an event loop. The current render cycle is one task. setTimeout(fn, 0) puts the function at the END of the task queue, AFTER the current render finishes. This gives Expo Router time to fully mount before you navigate.

```
# _layout.tsx — the full useEffect with cleanup
useEffect(() => {
if (!navigationState?.key) return;
// ^^ optional chaining: if navigationState is null, stop here
// .key existing = Expo Router is ready to navigate
const timeout = setTimeout(() => {
if (!isAuth) router.replace("/(auth)/login");
else router.replace("/(tabs)");
}, 0);
// 0ms = defer to next event loop tick
// Prevents "Attempted to navigate before mounting" crash
return () => clearTimeout(timeout);
// CLEANUP: if component unmounts before timeout fires,
// cancel it so it does not try to navigate to a gone screen
}, []); // run once on mount
```

#03 Context API

Sharing data without prop drilling

What is prop drilling and why is it a problem?

Prop drilling = passing data through multiple layers of components just to get it to the one that needs it. Every component in between receives a prop it does not use. In your app, user data needs to reach the login screen, the onboarding screen, the tab bar, and the profile screen. Without Context you would pass it through every layout and navigator in between.

```
# The 3-piece Context system — full picture
// PIECE 1: createContext creates the data pipe
const AuthContext = createContext<AuthContextType | undefined>(undefined);
// The pipe exists globally. undefined = default when no Provider above.
// PIECE 2: Provider injects data INTO the pipe
export const AuthProvider = ({ children }: { children: ReactNode }) => {
const [user, setUser] = useState<User | null>(null);
return (
<AuthContext.Provider value={{ user, signIn, signUp }}>
{children} // everything inside can read from the pipe
</AuthContext.Provider>
);
};
// PIECE 3: useContext reads FROM the pipe anywhere in the tree
export const useAuth = () => {
const context = useContext(AuthContext);
if (context === undefined) {
throw new Error("useAuth must be used within an AuthProvider");
}
return context; // returns { user, signIn, signUp }
};
// Usage in any screen:
const { signIn } = useAuth(); // login.tsx
const { signUp } = useAuth(); // signup.tsx
const { user } = useAuth(); // _layout.tsx (RootLayoutNav)
```

The Provider-before-Consumer crash — why _layout.tsx has two functions

useContext() searches UPWARD through the tree for the nearest Provider. If you call useAuth() in the same component that renders AuthProvider, the Provider has not mounted yet. useContext returns undefined. Destructuring { user } from undefined = crash. Fix: two functions. RootLayout renders the Provider. RootLayoutNav sits INSIDE it and safely calls useAuth().

```
# The two-function pattern from your actual _layout.tsx
// WRONG: useAuth() in same component that renders Provider
export default function RootLayout() {
const { user } = useAuth(); // CRASH — no Provider above this yet
return <AuthProvider>...</AuthProvider>;
}
// RIGHT: split into two functions
function RootLayoutNav() {
// Rendered INSIDE AuthProvider. Safe to call useAuth().
const { user } = useAuth();
}
export default function RootLayout() {
return (
<AuthProvider>
<GestureHandlerRootView>
<RootLayoutNav /> // useAuth() lives safely inside
</GestureHandlerRootView>
</AuthProvider>
);
}
```

#04 TypeScript

Interfaces, types, and why isAuth crashed you

What is this concept?

TypeScript is JavaScript with a type checker bolted on top. You describe the expected shape of your data using interfaces, and TypeScript checks at compile time — right in VS Code — whether you are using that data correctly. It catches entire categories of bugs before your code ever runs on a device.

```
# From AuthContext.tsx — interfaces annotated
export interface User {
id: string; // must be a string (Supabase UUID)
name: string;
email: string;
username: string;
profilePicture?: string; // ? = optional, can be undefined
onboardingCompleted: boolean;
}
interface AuthContextType {
user: User | null; // User object OR null (union type)
signUp: (email: string, password: string) => Promise<void>;
signIn: (email: string, password: string) => Promise<void>;
// Promise<void> = async function that returns nothing
}
// TypeScript enforces this at compile time.
// Try to access something NOT in the interface:
const { isAuth } = useAuth();
// ERROR: Property "isAuth" does not exist on type "AuthContextType"
// Caught before your code ever runs. That is the win.
```

TypeScript saved you from a 2-hour debugging session

Without TypeScript: const { isAuth } = useAuth() would silently set isAuth to undefined. undefined is falsy, so !isAuth is always true. The app always redirects to login, even when logged in. Zero error. Zero hint why. With TypeScript: instant red underline. Fixed in 5 seconds.

```
# Common TypeScript patterns from your codebase
// Union type: value can be one of multiple types
const [user, setUser] = useState<User | null>(null);
// Optional property
profilePicture?: string; // ok to omit from object
// Generic type parameter
createContext<AuthContextType | undefined>(undefined);
// Typed destructured prop
({ children }: { children: ReactNode })
// any type — escape hatch when type is unknown
} catch (error: any) { // error can be anything at runtime
```

#05 JSX

What it compiles to and why it has weird rules

What is this concept?

JSX looks like HTML inside your JavaScript but it is a completely different thing. Babel (the transpiler that runs before your app) converts every JSX tag into a plain JavaScript function call. By the time your code runs on Android, there is no JSX — only nested React.createElement() calls.

```
# What JSX actually compiles to
// What YOU write (JSX):
<View style={styles.container}>
<Text style={styles.title}>Hello</Text>
</View>
// What Babel converts it to:
React.createElement(
View,
{ style: styles.container },
React.createElement(Text, { style: styles.title }, "Hello")
)
// Every tag = function call
// Every attribute = property in the second argument object
// Every child = additional argument
```

```
# Curly braces in JSX — every usage from your code
// {} = "evaluate this as JavaScript"
<TextInput value={email} /> // pass a variable
<View style={styles.container}> // pass an object
<TouchableOpacity onPress={() => doIt()}> // pass a function
// Ternary conditional rendering:
{isLoading
? <ActivityIndicator size={24} color="#fff" />
: <Text>Sign Up</Text>
}
// You CANNOT use if/else directly inside JSX.
// Use ternary ? : or logical && instead:
{isLoading && <ActivityIndicator />} // renders only if true
```

> **React Native components are NOT HTML**
>
> View is not a div. Text is not a p tag. TextInput is not an input. They LOOK similar but they map to native Android/iOS UI widgets. View becomes android.view.View. Text becomes android.widget.TextView. ALL text must be inside a Text component. You cannot put bare strings in a View.

#06 async / await

Promises without the chaos

Why JavaScript cannot just wait

JavaScript is single-threaded — it can only do one thing at a time. If it froze and waited for every Supabase network request, your entire app would freeze. No scrolling. No animations. Frozen. Instead JavaScript uses Promises: start a task, let everything else keep running, call back when done. async/await is just cleaner syntax over Promises.

```
# async/await from your AuthContext.tsx — annotated
const signIn = async (email: string, password: string) => {
// async = this function returns a Promise.
// you are allowed to use await inside it.
const { data, error } = await supabase.auth.signInWithPassword(
{ email, password }
);
// await = pause HERE and wait for the Promise to resolve.
// The rest of the app keeps running. Only THIS function pauses.
// Supabase ALWAYS returns { data, error }.
// data.user = the logged-in user object on success
// error = Error object on failure, null on success
if (error) throw error;
// throw sends the error up to the nearest catch() block
if (data.user) {
console.log("Signed in:", data.user);
// MISSING: setUser() should be called here
}
};
```

```
# try / catch / finally — the error handling pattern
const handleSignUp = async () => {
setIsLoading(true);
try {
// Attempt this code. If anything throws, jump to catch.
await signUp(email, password);
router.replace("/(auth)/onboarding");
// Skipped if signUp() throws.
} catch (error: any) {
// Runs only if something inside try threw an error.
Alert.alert("Error", error.message || "Failed to sign up");
} finally {
// Runs no matter what. Success or failure.
setIsLoading(false); // always stop the spinner
// Without finally: if signUp() throws, spinner spins forever.
}
};
```

Supabase always returns { data, error } — check both

Supabase does not throw on failure by default — it returns an error object. If you only check data and ignore error, you might try to use data.user when data is null because an error occurred. Pattern: const { data, error } = await supabase... then if (error) throw error;

#07 Expo Router

File-based navigation — folders are routes

What is this concept?

Traditional React Native navigation required registering every screen manually in a config file. Expo Router takes a different approach inspired by Next.js: your file structure IS your navigation structure. The filename becomes the route. No config needed. _layout.tsx files control how the screens inside that folder are presented.

```
# Your folder structure = your navigation map
app/
_layout.tsx // ROOT layout — auth guard lives here
(auth)/
_layout.tsx // Stack navigator for auth flow
login.tsx // route: /(auth)/login
signup.tsx // route: /(auth)/signup
onboarding.tsx // route: /(auth)/onboarding
(tabs)/
_layout.tsx // Tab navigator for main app
index.tsx // route: /(tabs)/ (Home tab)
about.tsx // route: /(tabs)/about
profile.tsx // route: /(tabs)/profile
// () folders = Route Groups
// They are invisible in the URL — just organise related screens.
// _layout.tsx = defines HOW screens in that folder are shown.
// Stack = screens slide in from right, back gesture available.
// Tabs = bottom tab bar, tap to switch screens.
```

```
# push vs replace — when to use which
const router = useRouter();
// push() = navigate forward. User CAN go back.
router.push("/(auth)/signup");
// Use for: the Sign Up link on the login screen.
// User can swipe back to login.
// replace() = swap current screen. User CANNOT go back.
router.replace("/(auth)/login");
router.replace("/(tabs)");
// Use for: auth redirects.
// You should not be able to swipe back to a blank loading screen.
// useRootNavigationState — check if router is ready
const navigationState = useRootNavigationState();
if (!navigationState?.key) return; // not ready yet, stop
// navigationState?.key = optional chaining
// If navigationState is null/undefined, returns undefined (no crash)
```

The @/ path alias

In some files you used @/ for imports (e.g. import from '@/context/AuthContext'). @/ is a shortcut pointing to your project root. Without it you would write ../../context/AuthContext from nested files. The @/ version never breaks when you move files around.

Quick Cheat Sheet

Print this. Stick it somewhere visible. These are the 7 concepts you will use in every single React Native project including EarGuard.

Concept	One-liner	When you see it
`useState`	Persistent variable + re-render	const [x, setX] = useState(...)
`useEffect([])`	Run once on mount	Auth check, navigation guard
`useEffect([x])`	Run when x changes	Refetch when user.id changes
`createContext`	Create the data pipe	const Ctx = createContext(undefined)
`AuthProvider`	Inject data into pipe	<Ctx.Provider value={{user}}>
`useAuth()`	Read from pipe	const { user } = useAuth()
`interface`	TS object shape definition	interface User { id: string }
`async/await`	Wait for Promise without freezing	const { data } = await supabase...
`try/catch`	Handle async errors	catch (error: any) { Alert... }
`router.push`	Navigate forward (backable)	router.push('/(auth)/signup')
`router.replace`	Navigate (no going back)	router.replace('/(tabs)')
`?. chaining`	Null-safe property access	navigationState?.key

The Rules

1. useState = use it for ANY value the user changes. Email, password, loading state, image URI. All of it.

2. useEffect([]) = setup only. Never put navigation-after-user-action inside it.

3. Context Provider must be an ANCESTOR. Never call useAuth() in the same component that renders AuthProvider.

4. TypeScript errors with 'does not exist on type' = you accessed a property that is not in the interface. Fix the property name.

5. JSX {} = JavaScript. Everything inside is evaluated as code, not markup.

6. async/await: always wrap in try/catch/finally. Always check { data, error } from Supabase. Always stop spinners in finally.

7. router.replace for auth redirects. router.push for user navigation. Never confuse the two.

What comes next — Part 2

Part 2 is the practical half: every file in your codebase walked through line by line. AuthContext.tsx, both _layout.tsx files, login.tsx, signup.tsx, onboarding.tsx, profile.tsx — every decision explained and every bug annotated in context.

Spanish word of the day: depurar

depurar (deh-poo-RAR) = to debug. Example: Voy a depurar mi codigo. (I am going to debug my code.) Your most-used Spanish word going forward, guaranteed.